

Model validation in **ASP.NET Core MVC**

◆ 1. What is Model Validation?

Model validation is the process of verifying that the data submitted by a user (via forms, APIs, etc.):

- Is **complete** (required fields are filled)
- Is **correctly formatted** (email, date, number, etc.)
- Meets **business rules** (range, length, custom logic)

👉 In ASP.NET Core MVC, validation happens **automatically during model binding**.

◆ 2. Validation Flow in ASP.NET Core MVC

Here's the execution pipeline:

1. User submits form
2. Model Binding maps request data → Model object
3. Validation system checks model using rules
4. Validation results stored in:

ModelState

5. Controller checks:

```
if (ModelState.IsValid)
```

```
{
```

```
    // Process valid data
```

```
}
```

```
else
```

```
{
```

```
    // Return validation errors
```

```
}
```

◆ 3. Types of Model Validation

✓ 1. Server-Side Validation

- Happens on the server
- Always executed
- Secure and reliable

✓ 2. Client-Side Validation

- Happens in browser (JavaScript)
- Faster user feedback
- Uses **jQuery Unobtrusive Validation**

◆ 4. Data Annotations (Most Common Approach)

ASP.NET Core uses attributes from:

using System.ComponentModel.DataAnnotations;

Example Model

```
public class UserModel
```

```
{
```

```
    [Required(ErrorMessage = "Name is required")]
```

```
    public string Name { get; set; }
```

```
    [Range(18, 60, ErrorMessage = "Age must be between 18 and 60")]
```

```
    public int Age { get; set; }
```

```
    [EmailAddress(ErrorMessage = "Invalid Email")]
```

```
    public string Email { get; set; }
```

```
[StringLength(10, MinimumLength = 5)]  
  
public string Password { get; set; }  
  
}
```

◆ 5. Common Validation Attributes

Attribute	Purpose
[Required]	Field must not be empty
[StringLength]	Min & max length
[Range]	Numeric range
[EmailAddress]	Valid email format
[Phone]	Phone validation
[RegularExpression]	Custom pattern
[Compare]	Compare two fields (e.g., password confirm)

◆ 6. Controller Validation Handling

```
[HttpPost]  
  
public IActionResult Register(UserModel model)  
{  
    if (ModelState.IsValid)  
    {  
        return Content("Data is valid");  
    }  
  
    return View(model);  
}
```

```
}
```

◆ 7. Displaying Validation in Views (Razor)

◆ Field-level validation

```
<input asp-for="Name" />
```

```
<span asp-validation-for="Name"></span>
```

◆ Summary validation

```
<div asp-validation-summary="All"></div>
```

◆ 8. Client-Side Validation Setup

Ensure these scripts are included:

```
<script src="~/lib/jquery/jquery.js"></script>
```

```
<script src="~/lib/jquery-validation/jquery.validate.js"></script>
```

```
<script src="~/lib/jquery-validation-unobtrusive/jquery.validate.unobtrusive.js"></script>
```

👉 ASP.NET Core automatically generates validation rules into HTML using:

```
data-val="true"
```

◆ 10. Custom Validation

✅ Option 1: Custom Attribute

```
public class AdultAttribute : ValidationAttribute
```

```
{
```

```
    protected override ValidationResult IsValid(object value, ValidationContext context)
```

```
    {
```

```
        int age = (int)value;
```

```
        if (age < 18)
        {
            return new ValidationResult("Must be adult");
        }

        return ValidationResult.Success;
    }
}
```

Usage:

[Adult]

```
public int Age { get; set; }
```

◆ 11. Fluent Validation (Advanced)

Instead of attributes, you can use libraries like:

👉 FluentValidation

Example:

```
public class UserValidator : AbstractValidator<UserModel>
{
    public UserValidator()
    {
        RuleFor(x => x.Name).NotEmpty();
        RuleFor(x => x.Age).InclusiveBetween(18, 60);
    }
}
```

◆ 13. Validation vs Business Logic

⚠ Important distinction:

Validation	Business Logic
Checks input correctness	Applies domain rules
Happens early	Happens later
Example: Email format	Example: Email already exists

◆ 14. Best Practices

- ✓ Always validate on server
 - ✓ Use Data Annotations for simple rules
 - ✓ Use FluentValidation for complex scenarios
 - ✓ Never trust client-side validation alone
 - ✓ Keep validation logic separate from business logic
 - ✓ Return meaningful error messages
-